

Andromeda DS · Vue 3

Manual de integración para desarrolladores

 @andromeda/vue v0.1.0

 Vue 3.4+

 42 componentes

 Sincronizado con Figma

CONTENIDO

1. Qué es Andromeda
2. Requisitos previos
3. Instalación en 3 pasos
4. Verificar que funciona
5. Anatomía del Alert
6. Las 4 variantes
7. Props, slots y eventos
8. Recetas comunes
9. Buenas prácticas
10. Explorar los 42 componentes
11. Solución de problemas

1 Qué es Andromeda

Andromeda es la librería de componentes de INVEX. Un mismo diseño, dos implementaciones nativas —**Vue 3** y **React**— construidas *pixel perfect* desde el archivo de Figma “*Ui Kit Web*” y conectadas a él con Code Connect: cuando un diseñador selecciona un componente en Figma, ve el código real que debes usar.

42

componentes

1

archivo de tokens

0

colores
hardcodeados

100%

responsivo

Qué te da la librería

Componentes SFC (`<script setup>` + TypeScript) accesibles y responsivos, con las variantes y props exactas que definió diseño en Figma.

Cómo se distribuye

Como **código fuente** dentro del repositorio (no hay paquete en npm todavía), así que se consume apuntando a la carpeta del repo.

Estructura del repositorio:

```
Andromeda-ds/
├── vue/                ← @andromeda/vue (este manual)
│   └── src/
│       ├── components/ 42 componentes .vue
│       └── index.ts     punto de entrada
├── react/              ← @andromeda/react
├── tokens/
│   └── tokens.css       colores, espacios, tipografías (obligatorio)
└── README.md           documentación de cada componente
```

2 Requisitos previos

Requisito	Versión	Nota
Node.js	18 o superior	Para instalar dependencias
Vue	3.4+	Composition API con <code><script setup></code>
Bundler	Vite (recomendado) o Nuxt 3	Requiere <code>@vitejs/plugin-vue</code>
TypeScript	5.x (opcional)	Recomendado: la librería trae sus tipos



Los ejemplos usan **Vite**. Al final de la sección 3 están las notas para Nuxt 3.

3

Instalación en 3 pasos

1

Trae el repositorio a tu máquina

Clónalo **al lado** de tu proyecto (no dentro):

```
# quedando: /trabajo/mi-app y /trabajo/Andromeda-ds  
git clone https://github.com/victorrojas14/Andromeda-ds.git
```

Si tu equipo prefiere fijar la versión dentro del proyecto, usa un submódulo:

```
git submodule add https://github.com/victorrojas14/Andromeda-ds.git vendor/andromeda  
git submodule update --init --recursive
```

2

Apunta los alias a la librería

Así puedes importar con el nombre del paquete en lugar de rutas relativas largas.

 vite.config.ts

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { fileURLToPath } from 'node:url'

// ajusta la ruta a donde clonaste el repo
const DS = fileURLToPath(new URL('../Andromeda-ds', import.meta.url))

export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      '@andromeda/vue': DS + '/vue/src',
      '@andromeda/tokens': DS + '/tokens',
    },
  },
})
```

Y para que TypeScript resuelva los tipos:

 tsconfig.json

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@andromeda/vue": ["../Andromeda-ds/vue/src"],
      "@andromeda/vue/*": ["../Andromeda-ds/vue/src/*"]
    }
  }
}
```

3

Carga los tokens y la tipografía

Una sola vez en el punto de entrada de tu app. Sin esto los componentes se ven sin color ni espaciado, porque todo el DS lee variables CSS.

src/main.ts

```
import { createApp } from 'vue'
import '@andromeda/tokens/tokens.css' ← obligatorio, antes de tus estilos
import App from './App.vue'

createApp(App).mount('#app')
```

La tipografía del DS es **Poppins**; agrégala en tu HTML:

index.html

```
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Poppins:wght@
```



Nuxt 3: define el alias en `nuxt.config.ts` con `alias: { '@andromeda/vue': ... }`, agrega el CSS en `css: ['@andromeda/tokens/tokens.css']` y añade el paquete a `build.transpile` para que Nitro procese los `.vue`.

4

Verificar que funciona

Pega esto en cualquier vista. Si ves la alerta verde con su icono, la instalación está lista:

src/App.vue

```
<script setup>
import { Alert } from '@andromeda/vue/components/Alert'
</script>

<template>
  <Alert variant="success" message="¡Listo!">
    Tu operación se registró correctamente.
  </Alert>
</template>
```

RESULTADO ESPERADO



¡Listo!

Tu operación se registró correctamente.



Fíjate en el import: `@andromeda/vue/components/Alert` en lugar de `@andromeda/vue`. Es lo recomendado y en la sección 9 explicamos por qué.

5

Anatomía del Alert

El Alert se arma con cinco piezas; todas son opcionales salvo el contenido:

PARTES DEL COMPONENTE



2

Aviso

3

Texto de la alerta que explica la situación.

4

Acción



- 1 Icono** — se elige solo según el `variant`. Slot `#icon` para cambiarlo, prop `hide-icon` para quitarlo. (es el círculo a la izquierda)
- 2 Mensaje** — texto corto en negrita con divisor. Prop `message`.
- 3 Contenido** — el cuerpo. Va en el **slot por defecto**.
- 4 Acción** — cualquier elemento a la derecha, normalmente un `Button`. Slot `#action`.
- 5 Cerrar** — visible por defecto. Se apaga con `:closable="false"`. (la `×` al extremo derecho)

6

Las 4 variantes

La prop `variant` cambia color de fondo, color de texto e icono, todo al mismo tiempo. Son los cuatro estados semánticos definidos en Figma:

VARIANT="SUCCESS" · VARIANT="WARNING" · VARIANT="DANGER" · VARIANT="INFO"



¡Bien hecho!

La transferencia se aplicó con éxito.



¡Alerta!

Tu sesión expira en 2 minutos.



Error

No pudimos procesar el movimiento.



Aviso

Actualizamos nuestros términos y condiciones.



CÓDIGO

```
<Alert variant="success" message="¡Bien hecho!">La transferencia se aplicó con éxito.</Alert>
<Alert variant="warning" message="¡Alerta!">Tu sesión expira en 2 minutos.</Alert>
<Alert variant="danger" message="Error">No pudimos procesar el movimiento.</Alert>
<Alert variant="info" message="Aviso">Actualizamos nuestros términos y condiciones.</Alert>
```



Si omites `variant`, el componente usa `"success"`. Cambiar de variante es **una sola palabra**: nunca toques colores a mano.

7

Props, slots y eventos

Props

Prop	Tipo	Default	Para qué sirve
<code>variant</code>	<code>'success' 'warning' 'danger' 'info'</code>	<code>'success'</code>	Estado semántico: define fondo, color de texto e icono.
<code>message</code>	<code>string</code>	—	Texto corto en negrita, separado del cuerpo por un divisor.
<code>hide-icon</code>	<code>boolean</code>	<code>false</code>	Oculto el icono de la izquierda.
<code>closable</code>	<code>boolean</code>	<code>true</code>	Muestra u oculta el botón de cerrar.

Slots

Slot	Para qué sirve
<code>default</code>	Cuerpo de la alerta.
<code>#icon</code>	Reemplaza el icono del variant por el tuyo.
<code>#action</code>	Elemento a la derecha, normalmente un <code>Button</code> .

Eventos

Evento	Cuándo se emite
<code>@close</code>	Al pulsar la × (el componente ya se oculta solo).

AlertBlock — la versión en bloque

Para mensajes más largos, con título propio y un texto final separado por un divisor:

¡Info!

Tu contrato se renovará automáticamente el 1 de marzo con las mismas condiciones vigentes.

Puedes cancelarlo en cualquier momento desde Configuración.

CÓDIGO

```
<AlertBlock variant="info">
  Tu contrato se renovará automáticamente el 1 de marzo con las mismas condiciones vigentes
<template #footer>
  Puedes cancelarlo en cualquier momento desde Configuración.
</template>
</AlertBlock>
```

Prop / Slot	Tipo	Default	Nota
<code>variant</code>	<code>'success' 'warning' 'info'</code>	<code>'success'</code>	No tiene <code>danger</code> (así está en Figma).
<code>title</code>	<code>string</code>	según variant	"¡Bien hecho!", "¡Alerta!" o "¡Info!".
<code>default</code> (slot)	—	—	Cuerpo del mensaje.
<code>#footer</code> (slot)	—	—	Texto final tras el divisor.

8 Recetas comunes

Con un botón de acción

**Aviso**

Tienes un documento pendiente de firma.

[Ver documento](#)

```
<script setup>
import { Alert } from '@andromeda/vue/components/Alert'
import { Button } from '@andromeda/vue/components/Button'
</script>

<template>
  <Alert variant="info" message="Aviso">
    Tienes un documento pendiente de firma.
    <template #action>
      <Button variant="primary" appearance="outline" size="sm">Ver documento</Button>
    </template>
  </Alert>
</template>
```

Sin icono y sin botón de cerrar

```
<!-- hide-icon quita el icono; :closable="false" quita la X -->
<Alert variant="warning" hide-icon :closable="false">
  Revisa los datos antes de continuar.
</Alert>
```

Reaccionar al cierre

```
<!-- El Alert se oculta solo; @close sirve para tu lógica -->
<Alert variant="success" @close="registrarEvento('alerta_cerrada')">
  Guardamos tus cambios.
</Alert>
```

Mostrar una alerta según el resultado de una operación

```

<script setup>
import { ref } from 'vue'
import { Alert } from '@andromeda/vue/components/Alert'

const estado = ref(null) // 'ok' | 'error' | null
</script>

<template>
  <Alert v-if="estado === 'ok'" variant="success" message="¡Listo!">
    Operación completada.
  </Alert>
  <Alert v-else-if="estado === 'error'" variant="danger" message="Error">
    Intenta de nuevo.
  </Alert>
</template>

```

9 Buenas prácticas

Importa por componente, no del paquete completo

Medido en un proyecto Vite real con este mismo Alert:

Forma de importar	CSS final	Assets extra
<code>from '@andromeda/vue'</code>	111.8 KB	+263 KB de logos
<code>from '@andromeda/vue/components/Alert'</code> ← recomendado	15.9 KB	ninguno

El CSS de cada componente se importa junto con su código, y las hojas de estilo *no* se pueden depurar automáticamente. Importando por subpath te llevas **solo lo que usas**: 7 veces menos CSS.

Nunca escribas colores ni medidas a mano

```

/* ❌ mal: se desincroniza cuando diseño cambia la paleta */
.mi-caja { background: #a41d36; padding: 20px; }

/* ✅ bien: usa los tokens del DS */
.mi-caja { background: var(--color-primary); padding: var(--space-20); }

```

Si diseño cambia el rojo INVEX en Figma, se actualiza `tokens.css` y tu pantalla se actualiza sola. Con un hex escrito a mano, no.

No edites la librería desde tu proyecto

Si un componente no cubre tu caso, envuélvelo o pide el cambio en el DS. Editar `Andromeda-ds/` desde una app rompe a los demás equipos y se pierde en el próximo `git pull`.



Para personalizar detalles puntuales pasa tu propia clase: `<Alert class="mi-espaciado" />`
— Vue la suma a las clases del DS automáticamente.

10 Explorar los 42 componentes

La referencia viva es **Storybook**: cada componente con todas sus variantes y un panel para cambiar props en caliente. Desde el repositorio del DS:

```
cd Andromeda-ds/vue
npm install
npm run storybook    → http://localhost:6007
```

Además, el `README.md` del repositorio documenta cada componente con su origen en Figma, y en **Figma → Dev Mode** cualquier componente muestra el snippet de código listo para copiar (Code Connect); elige la pestaña **Vue**.

11 Solución de problemas

Síntoma	Causa	Solución
Los componentes salen sin colores ni espacios	No se cargó <code>tokens.css</code>	Importa <code>'@andromeda/tokens/tokens.css'</code> en <code>main.ts</code> (paso 3)
Se ve con otra tipografía	Falta Poppins	Agrega el <code><link></code> de Google Fonts en tu <code>index.html</code>
<code>Failed to resolve import "@andromeda/vue"</code>	Alias mal configurado o ruta equivocada	Revisa la ruta del alias en <code>vite.config.ts</code> ; debe apuntar a <code><repo>/vue/src</code>
<code>Could not load ../tokens.css</code>	La ruta relativa del alias no resuelve	Verifica cuántos niveles sube tu <code>'../Andromeda-ds'</code> o usa ruta absoluta para probar
Los <code>.vue</code> del DS no compilan	Falta el plugin de Vue	Asegúrate de tener <code>@vitejs/plugin-vue</code> en <code>plugins: [vue()]</code>
En Nuxt marca error al hacer SSR	El paquete no se transpila	Agrega <code>'@andromeda/vue'</code> a <code>build.transpile</code> en <code>nuxt.config.ts</code>
TypeScript no encuentra los tipos	Faltan los <code>paths</code>	Agrega <code>paths</code> en <code>tsconfig.json</code> (paso 2)